

CS3611 Project 9

DDoS 智能检测与自动防御系统

大作业讲解



一、项目概述

本项目构建了一套完整的 DDoS

攻击智能检测与自动防御系统，涵盖攻击模拟、流量采集、特征提取、MLP 模型推理、iptables 自动封禁及效果对比全链路。系统支持普通演示（防御前后对比）和实时攻防演示（攻击进行中在线联动）两种模式。

核心能力

能力	实现方式
攻击模拟	SYN Flood、HTTP Flood、UDP 反射放大，混合同行
流量采集	tcpdump 抓包，记录 PPS / 协议分布 / IP 熵
智能检测	PyTorch MLP + KMeans + AutoEncoder，三模型 SOTA 融合 F1=0.9997
自动防御	iptables 限速 + 黑名单 DROP + hashlimit 限速，模型驱动自动封禁
效果对比	防御前后同参攻击，量化流量与 CPU 负载变化
实时联动	攻击进行中按 4s 窗口循环抓包、推理、封禁

安全约束：所有攻击脚本限定

127.0.0.0/8、10.0.0.0/8、172.16.0.0/12、192.168.0.0/16，禁止公网目标。推理阶段自动丢弃公网 IP。

二、系统架构

模块分层

层次	目录	主要组件
整合层	scripts/	run_demo.sh (普通)、run_realtime_demo.sh (实时)
攻击层	attacks/	syn_flood.py、http_flood.py、udp_reflection_sim.py
采集层	—	tcpdump + 受害 HTTP 服务 (127.0.0.1:8080)
特征层	features/	extract_features.py → 8 维特征 CSV
模型层	models/	infer.py (MLP) + KMeans + AutoEncoder + SOTA 融合
防御层	defense/	iptables_rules.sh、apply_decision.py、block_ip.sh
存储层	storage/	redis_store.py (Redis Stream 双写)

普通演示数据流

攻击前抓包 → 离线 MLP 推理与封禁 → 防御后再次攻击，展示防御前后流量对比。

```

正常流量 10s → 混合攻击 20s → tcpdump 抓包
→ extract_features → 特征 CSV
→ infer.py (MLP) → decision.json
→ apply_decision.py → block_ip.sh → iptables
→ 同参攻击复测 → 对比流量变化

```

实时攻防演示数据流

混合攻击在前台持续进行时，后台按 4s 窗口循环执行抓包→推理→封禁。

```

前台: run_mixed_attack.sh 持续 120s
后台每 4s: tcpdump → extract_features
→ infer.py → apply_decision → iptables

```

三、攻击模拟模块

组件	攻击类型	实现方式
syn_flood.py	SYN Flood	TCP SYN 洪泛, 伪造 127.x 源 IP
http_flood.py	HTTP Flood	高频率 GET/POST, 记录成功率
udp_reflection_sim.py	UDP 反射放大	模拟反射放大, 小请求换大响应
run_mixed_attack.sh	混合攻击	并行启动 SYN + HTTP + UDP
normal_traffic.py	正常流量	模拟正常 HTTP 请求建立基线

四、流量采集与特征提取

采集方式

使用 tcpdump 在 loopback 接口抓包, 按窗口时长保存 PCAP。支持 CAPTURE_IFACE=any。

八维统计特征

特征	物理含义	正常值	攻击值
pps	每秒包数, 洪泛强度	~60	1000+
bps	每秒比特数, 带宽占用	低	高
syn_count	SYN 包数量	低	SYN Flood 极高
syn_ack_ratio	SYN/ACK 比值, 连接异常	~1	显著升高
unique_src_ips	窗口内不同源 IP 数	1	数十 (伪造多源)
ip_entropy	源 IP 香农熵	低 ~0	高 (均匀分布)
avg_pkt_size	平均包长 (字节)	较稳定	随协议波动

特征提取管线: PCAP -> extract_features.py (1s 窗口聚合) -> 8 维特征 CSV。训练前经 StandardScaler 标准化, Scaler 参数随模型保存。

五、智能检测模型

MLP 二分类器

基于 PyTorch 的三层感知机 (8->32->16->1), ReLU 激活 + Dropout。BCEWithLogitsLoss 引入正类权重解决类别不平衡。推理时 Sigmoid 得概率, 阈值默认 0.80。

decision.json 输出格式

字段	含义
src_ip	待处置源 IP（仅私网地址）
confidence	模型置信度（0~1）
action	处置动作（block）
reason	判定理由 + 攻击类型

实测推理结果（Run report_demo_01）

指标	数值
决策总数	1470
攻击判定（conf>=0.80）	1470（100%）
置信度范围	0.835 ~ 1.000
平均置信度	0.985

无监督异常检测

除 MLP 外还实现了 K-Means 聚类（轮廓系数 0.888）和 AutoEncoder 重构检测。三模型经 SOTA 融合后 F1=0.9997。详见《模型详解》PDF。

六、自动防御机制

防御链路

```
decision.json -> apply_decision.py -> block_ip.sh -> iptables
```

四层防御策略

机制	规则内容
基础限速	SYN <= 50/s, HTTP <= 120/s, 异常 TCP flag 丢弃
loopback 限速	127.0.0.0/8 源 IP -> hashlimit 令牌桶限速
黑名单 DROP	私网源 IP (10.x/172.16.x/192.168.x) -> 直接丢弃
模型驱动封禁	MLP 推理结果自动触发 block_ip.sh, 全自动闭环

七、两种演示模式

模式对比

对比项	普通演示	实时攻防演示
入口脚本	run_demo.sh	run_realtime_demo.sh
封禁时机	攻击结束后离线推理	攻击进行中在线推理封禁
决策粒度	单次 decision.json	多窗口合并
核心价值	量化证明防御有效	展示 AI+防火墙实时联动

普通演示时序

阶段	动作	产出
A	正常流量 10s + 抓包	normal.pcap / csv
B	混合攻击 20s + 抓包/推理/防御	attack_before / decision.json
C	同参攻击 20s + 抓包	attack_after (对比用)

实时演示时序

步骤	执行方	动作
1	脚本	启动受害服务 + 正常基线抓包
2	前台	run_mixed_attack.sh 持续攻击 120s
3	后台	每 4s 抓包->推理->封禁
4	收尾	合并窗口决策, 输出摘要 CSV

八、测试结果

防御前后流量对比（核心证据）

场景	包数	时长(s)	平均 PPS
正常流量	588	9.98	58.9
攻击（防御前）	29,300	24.23	1209.1
攻击（防御后）	3,450	22.68	152.2

指标	防御前	防御后	降幅
总包数	29,300	3,450	88.2%
平均 PPS	1209.1	152.2	87.4%
CSV 峰值 PPS	1475	411	72.1%

CPU / 内存负载变化

指标	防御前	防御后	变化
CPU 0	62.2%	33.8%	降约一半
CPU 1	81.8%	35.7%	双核均显著降低
内存	1.01 GB	1.01 GB	基本不变
攻击进程峰值	86.5%	合计较低	被有效压制

实时攻防联动结果

指标	数值
完成窗口数	3/3 (extract/infer/apply 均 ok)
窗口决策总数	616 条
最高置信度	1.000
首封延迟	约 46 s
攻防峰值	2.0K PPS

结论：开启智能防御后同等攻击下包数降 88.2%、PPS 降 87.4%、CPU 降约一半。实时模式 3 个 4s 窗口均完成全链路，证明系统具备在线实时防御能力。

九、扩展功能完成情况

作业要求共列 4 项可选扩展（满分 75 分，上限 25 分）。本组全部实现且每项有独立代码和测试验证。

扩展项	分值	实现要点	核心证据
智能攻击分类器	25	MLP 8→32→16→1, 833 参数	decision.json 均值 0.985
无监督异常检测	25	KMeans + AutoEncoder 双模型	SOTA 融合 F1=0.9997
自动化防御	10	检测→阻断全自动闭环	defense_blocks.log 616 条
CDN 集成	15	Nginx 反代 + 限速 + 缓存	defense/nginx.conf
Redis 存储（自定义）	—	特征/决策/封禁双写 Stream	redis-cli XRANGE 可查

十、运行指南

环境要求

操作系统：Ubuntu 20.04+（需 bash/sudo/tcpdump/iptables）。Python 3.8+，使用项目 .venv。网络限定 loopback/私网。Redis 可选（Docker 启动）。

一键运行

```
# 1. 安装
sudo apt install -y python3 python3-pip python3-venv tcpdump iptables curl
python3 -m venv .venv && .venv/bin/pip install -r models/requirements.txt
# 2. 配置
cp scripts/demo.env.example scripts/demo.env # 确认 TARGET_IP=127.0.0.1
# 3. 自检
bash scripts/check_group_contract.sh attack|defense|model
bash scripts/run_demo.sh --check-only
# 4. 普通演示
bash scripts/run_demo.sh --run-id report_demo_01
# 5. 实时攻防
bash scripts/run_realtime_demo.sh --run-id realtime_demo_01
# 6. 可视化
.venv/bin/python scripts/visualize_demo.py --run-id report_demo_01
# 7. 清理
bash defense/unblock_all.sh --project-tag cs3611-ddos
```

产出目录

data/pcap/<run_id>/	PCAP 抓包
data/features/<run_id>/	特征 CSV
data/logs/<run_id>/	decision.json / 防御日志 / 可视化 HTML

十一、项目亮点

全链路闭环自动化

攻击→抓包→特征→AI推理→iptables封禁→效果验证，一键运行，端到端可复现。

双模式覆盖

离线对比（量化防御效果）+ 在线联动（实时攻防），各有独立脚本入口，覆盖不同演示场景。

三模型融合检测

MLP(有监督) + KMeans + AutoEncoder(无监督)，SOTA 融合 F1=0.9997，非单一模型凑数。

防御效果硬核量化

包数降 88.2%，PPS 降 87.4%，CPU 降约一半。不靠“看起来有用”，靠“测出来有用”。

扩展项全覆盖

4 项作业扩展 + 自定义 Redis 全部实现含代码，每项可独立运行验证。

安全设计严谨

攻击限定私网 + 推理自动过滤公网 IP + 演示前自动清理旧规则，三层安全防护。

工程化程度

contract 自检、demo.env 配置、Redis 双写、可视化仪表盘、完整操作手册。